Programming Fundamentals II: Assignment 3

Due Date: Saturday, November 30 2024 at Midnight

Part 1. Cargo Rearranging (20 Marks)

Suppose we want to build a train using a recursive data structure. We will use a Java class to track each individual car of the train. Each TrainCar will track its name (String), its cargo (String) and its load (double, in 100s of kg). We want to not only be able to store and track everything being carried by this train, but also rearrange the cargo to our liking. Download the Driver class provided.

1. **(3 Mark**s) Write the code for the TrainCar class. Include any getters and setters you will need. Include only the constructor required by the Driver class. The toString for the train class should return its name, then its cargo and load.

2. **(2 Marks)** Add a method called swapCargo to your TrainCar class that will accept another TrainCar as its input. The method will then swap the contents of the two cars (the cargo and load).

3. **(3 Marks)** Implement the natural ordering for TrainCars. Our desired natural ordering for TrainCars is by the weight of their loads, ascending.

4. **(3 Marks)** Add a void method to your TrainCar class called printTrain. When called on a TrainCar, the method should print out that car, then recursively print out each car that follows it in the train.

5. **(5 Marks)** Complete the selectionSort method in the Driver class. This method should accept the head of a train as input (the first TrainCar). It will then use a modified selectionSort algorithm to sort the train cargos into their *natural ordering*. The train cars will not change order, we are just going to swap cargos between the cars so they are in order. See the sample output.
   To get full marks, accomplish this problem using recursion. That is, without putting your TrainCars into an array/ArrayList.
   *Note: You can modify the selectionSort code provided in the slides. You might find it difficult to use the **for** loops. Consider another strategy if you're having trouble.*

6. **(2 Marks)** Create an *alternate ordering* for TrainCar called OrderByCargo that can be used to sort the train cars alphabetically (lexicographically) by the names of their cargos.

7. **(2 Marks)** Overload the selectionSort method so that it will also accept Comparator object and use that object to sort the train cars instead of the natural order.

```
Sample output:

Unsorted:
Car A: wheat (100.0)
Car B: oats (120.0)
Car C: gold (40.0)
Car D: pork (50.0)
Car E: coal (200.0)
Car F: canola (45.0)
Car G: one really big egg (150.0)

Natural Order:
Car A: gold (40.0)
Car B: canola (45.0)
Car C: pork (50.0)
Car D: wheat (100.0)
Car E: oats (120.0)
Car F: one really big egg (150.0)
Car G: coal (200.0)

Alternate Order:
Car A: canola (45.0)
Car B: coal (200.0)
Car C: gold (40.0)
Car D: oats (120.0)
Car E: one really big egg (150.0)
Car F: pork (50.0)
Car G: wheat (100.0)
```

Add the **date and your name and student number** to the top of your java files. Please submit your java and files to Nexus before the end of the day on Saturday, December 02 at Midnight. **Late submissions will be not be accepted.**